

Лабораторная №7. Добавление поддержки интерфейса SPI

Цель работы

Изучить принципы работы SPI, настроить контроллер Allwinner D1 в Device Tree, включить необходимые драйверы ядра Linux и проверить обмен с SPI-устройством из пользовательского пространства.

Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с подготовленной SD-картой;
- компьютер со средой сборки ядра Linux и исходными файлами Device Tree;
- результаты лабораторных №4–6;
- USB-UART-преобразователь с логическими уровнями 3,3 В;
- перемычка для теста MOSI–MISO;
- статически собранная для RISC-V утилита `spidev_test` из исходников Linux 6.16.

Опасность повреждения

Все внешние соединения с GPIO выполняйте только при выключенном питании платы. Выводы Allwinner D1 работают с логическими уровнями 3,3 В; подача 5 В может повредить плату.

Ключевые понятия

Определение

SPI (Serial Peripheral Interface) — синхронный последовательный интерфейс, в котором контроллер формирует тактовый сигнал и выбирает периферийное устройство отдельной линией CS.

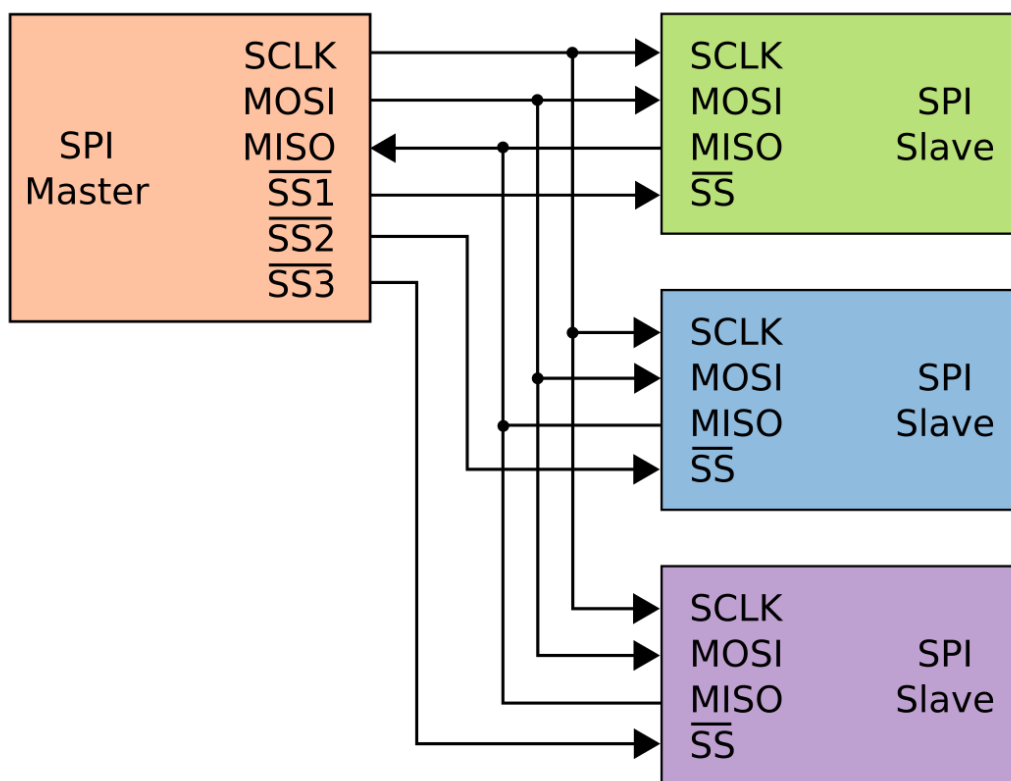


Рис. 1 – Топология SPI с общими линиями данных и отдельными сигналами выбора периферийных устройств

SPI использует четыре основных сигнала:

- SCK — тактовый сигнал;
- MOSI — данные от контроллера к периферийному устройству;
- MISO — данные от периферийного устройства к контроллеру;
- CS — выбор конкретного периферийного устройства.

Линии MOSI, MISO и SCK могут быть общими для нескольких устройств, а линия CS должна быть отдельной для каждого из них. Специализированный вывод контроллера SPI или обычный GPIO может формировать CS.

Определение

Полнодуплексный обмен — одновременная передача данных по MOSI и приём данных по MISO на каждом такте SCK.

Каждая операция SPI принимает столько же байтов, сколько передаёт, даже если принятые байты не содержат полезной информации.

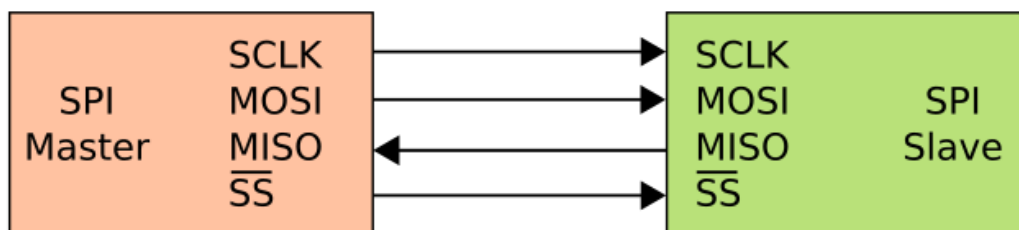


Рис. 2 – Подключение одного SPI-устройства к Lichee RV Dock

Порядок выполнения

Анализ аппаратной конфигурации

На Lichee RV Dock сигналы SPI1 выведены на контакты PD10–PD15 как альтернативные функции GPIO.

Контакт разъёма	Линия SoC	Сигнал SPI1	Назначение в работе
21	PD10	CS0	аппаратный выбор устройства
22	PD11	SCK	тактовый сигнал
19	PD12	MOSI	передаваемые данные
17	PD13	MISO	принимаемые данные
9	PD14	HOLD	в loopback не используется
15	PD15	WP	в loopback не используется

Sipeed Lichee RV DOCK pinout diagram

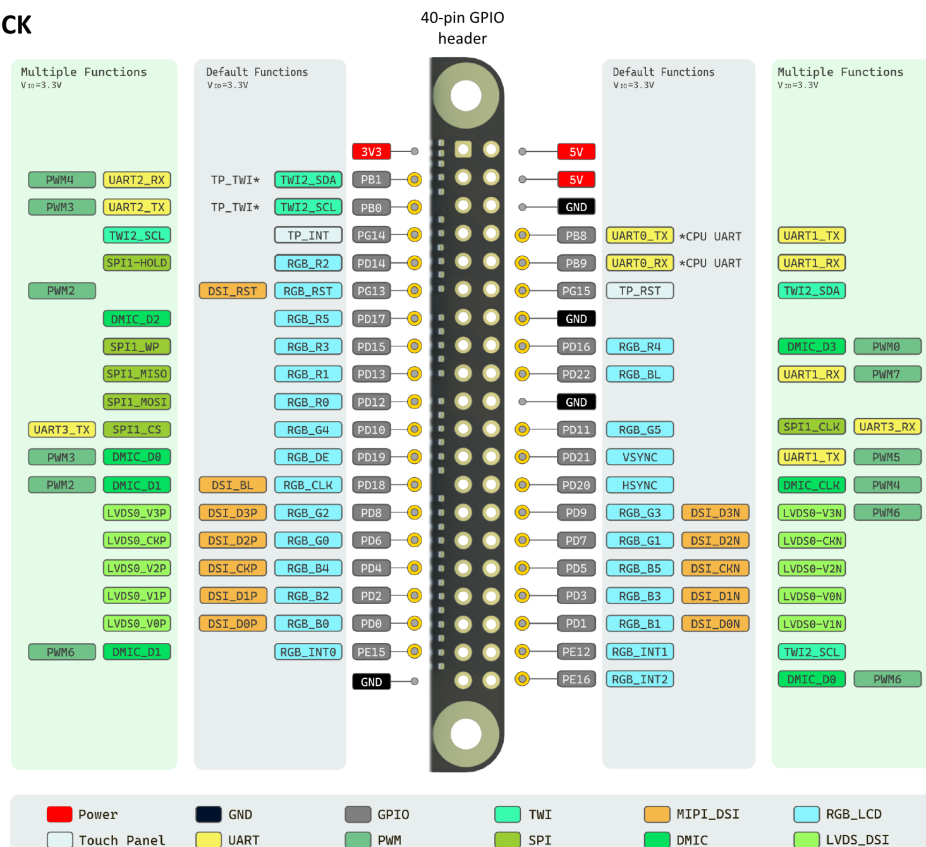


Рис. 3 – Распиновка Lichee RV Dock: сигналы SPI1 на выводах PD10–PD15

По умолчанию эти выводы могут быть назначены RGB-интерфейсу дисплея. Перед активацией SPI проверьте исходный Device Tree и исключите конфликт функций.

Внимание

Один физический вывод нельзя одновременно использовать для RGB и SPI. Если узел дисплея занимает PD10–PD15, отключите конфликтующую функцию в Device Tree до включения SPI1.

Анализ описания SPI в Device Tree

Исходные файлы платы находятся в каталоге arch/riscv/boot/dts/allwinner/. Файл sun20i-d1-lichee-rv-dock.dts включает описание базовой платы, а оно, в свою очередь, включает общие файлы .dtsi с контроллерами SoC.

Определение

Device Tree Source Include (DTSI) — включаемый исходный файл дерева устройств с общими описаниями аппаратных блоков и выводов.

В общем файле объявлены группы выводов двух контроллеров SPI:

```
/omit-if-no-ref/
spi0_pins: spi0-pins {
    pins = "PC2", "PC3", "PC4", "PC5";
```

```

        function = "spi0";
};

/omit-if-no-ref/
spi1_pb_pins: spi1-pb-pins {
    pins = "PB0", "PB8", "PB9", "PB10", "PB11", "PB12";
    function = "spi1";
};

/omit-if-no-ref/
spi1_pd_pins: spi1-pd-pins {
    pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
    function = "spi1";
};

```

Директива `/omit-if-no-ref/` исключает неиспользуемый узел из итогового DTB. Контроллеры объявлены отдельно и изначально отключены:

```

spi0: spi@4025000 {
    compatible = "allwinner,sun20i-d1-spi",
                "allwinner,sun50i-r329-spi";
    reg = <0x4025000 0x1000>;
    interrupts = <31 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI0>;
    dmas = <&dma 22>, <&dma 22>;
    dma-names = "rx", "tx";
    num-cs = <1>;
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
};

spi1: spi@4026000 {
    compatible = "allwinner,sun20i-d1-spi-dbi",
                "allwinner,sun50i-r329-spi-dbi",
                "allwinner,sun50i-r329-spi";
    reg = <0x4026000 0x1000>;
    interrupts = <32 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI1>, <&ccu CLK_SPI1>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI1>;
    dmas = <&dma 23>, <&dma 23>;
    dma-names = "rx", "tx";
    num-cs = <1>;
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
};

```

Свойство `compatible` связывает аппаратный узел с поддерживающим его драйвером. При обнаружении активного совместимого узла драйвер выполняет инициализацию устройства.

Активация SPI1 и SPIDEV

Чтобы сохранить базовый DTS как резервный, создайте отдельный файл `sun20i-d1-lichee-rv-dock-spi-lab.dts`:

```
#include "sun20i-d1-lichee-rv-dock.dts"

/ {
    aliases {
        spi1 = &spi1;
    };
};

&pio {
    /omit-if-no-ref/
    spi1_pd_lab_pins: spi1-pd-lab-pins {
        pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
        function = "spi1";
    };
};

&spi1 {
    pinctrl-0 = <&spi1_pd_lab_pins>;
    pinctrl-names = "default";
    status = "okay";

    spidev@0 {
        compatible = "rohm,dh2228fv";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

Alias фиксирует номер контроллера как `spi1`, а отдельное имя `pinctrl`-группы не требует изменения общего `.dtsi`. Для доступа из пользовательского пространства дочерний узел использует идентификатор из таблицы SPIDEV.

Внимание

Идентификатор `rohm,dh2228fv` применяется здесь как учебный способ привязать SPIDEV. Для реального периферийного устройства используйте его штатный `compatible` и драйвер, если они существуют.

Настройка ядра

Используйте конфигурацию, подготовленную в лабораторной №4. Проверьте и включите параметры:

```
CONFIG_SPI=y
CONFIG_SPI_MASTER=y
CONFIG_SPI_SUN6I=y
CONFIG_SPI_SPIDEV=y
CONFIG_DMA_SUN6I=y
```

Драйвер SPI_SUN6I исторически назван по первому поддерживаемому семейству, но применяется и к совместимым контроллерам новых SoC, включая sun20i. DMA_SUN6I предоставляет DMA-каналы для длинных передач и снижает участие CPU в обмене, но не является обязательным условием для работы: драйвер SPI умеет работать через FIFO и прерывания. В Linux 6.16 передачи длиной до 64 байт включительно выполняются без DMA; передача большей длины может использовать DMA, если оба канала успешно выделены.

Внимание

Рабочая конфигурация требует согласованных изменений Device Tree и ядра. Без SPI_SUN6I контроллер не работает, без SPI_SPIDEV не появляется символьное устройство в каталоге /dev, а без DMA_SUN6I длинные передачи переходят на программный путь вместо DMA.

Добавьте новый DTB в arch/riscv/boot/dts/allwinner/Makefile:

```
dtb-$(CONFIG_ARCH_SUNXI) += sun20i-d1-lichee-rv-dock-spi-lab.dtb
```

Скопируйте проверенную конфигурацию лабораторной №4 в новый каталог сборки, примените зависимости Kconfig, соберите ядро и DTB и проверьте, что получены оба файла:

```
KERNEL_SRC="$HOME/kernel-source-6.16"
BUILD_DIR="$HOME/kernel-out-spi"

mkdir -p "$BUILD_DIR"
cp "$HOME/kernel-out-6.16/.config" "$BUILD_DIR/.config"
make -C "$KERNEL_SRC" O="$BUILD_DIR" ARCH=riscv \
    CROSS_COMPILE=riscv64-linux-gnu- olddefconfig
make -C "$KERNEL_SRC" O="$BUILD_DIR" ARCH=riscv \
    CROSS_COMPILE=riscv64-linux-gnu- -j"$(nproc)" Image dtbs
ls -lh "$BUILD_DIR/arch/riscv/boot/Image"
ls -lh "$BUILD_DIR/arch/riscv/boot/dts/allwinner/"*spi-lab.dtb
```

Скопируйте комплект на раздел BOOT под отдельными именами. В extlinux/extlinux.conf оставьте также запись с ранее проверенными Image и DTB. Корневой раздел лучше задавать через фактический

PARTUUID, поскольку после активации дополнительных устройств порядок /dev/mmcblkN может измениться:

```
default spi-lab

label spi-lab
    kernel /Image-6.16.0-spi-lab
    fdt /sun20i-d1-lichee-rv-dock-spi-lab.dtb
    append root=PARTUUID=<PARTUUID_ROOTFS> rootwait loglevel=7
    ↪ console=ttyS0,115200

label known-good
    kernel /Image
    fdt /sun20i-d1-lichee-rv-dock.dtb
    append root=PARTUUID=<PARTUUID_ROOTFS> rootwait loglevel=7
    ↪ console=ttyS0,115200
```

Значение PARTUUID получите командой blkid для раздела ROOTFS конкретной карты.

Соберите тестовую утилиту на рабочем компьютере и скопируйте её в ROOTFS:

```
riscv64-linux-gnu-gcc -O2 -Wall -static \
-o spidev_test tools/spi/spidev_test.c
file spidev_test
```

Проверка устройства

После загрузки проверьте наличие символического устройства:

```
uname -r
ls -l /dev/spidev*
dmesg | grep -i spi
readlink -f /sys/bus/spi/devices/spi1.0/driver
readlink -f /sys/class/spi_master/spi1/device/driver
mountpoint -q /sys/kernel/debug || mount -t debugfs debugfs /sys/kernel/debug
grep -E '4026000\.spi:(tx|rx)' /sys/kernel/debug/dmaengine/summary
```

Проверка

Ожидаемый результат. Для шины 1 и устройства с reg = <0> появляется /dev/spidev1.0. Фактическое имя подтвердите по выводу системы.

Полнодуплексный тест

Соедините короткой перемычкой **контакт 19 (PD12/MOSI) с контактом 17 (PD13/MISO)**. После повторной загрузки создайте детерминированный файл из четырёх последовательностей 0–255 и проверьте обмен на трёх частотах:

```
python3 - <<'PY'
from pathlib import Path

Path("tx-1024.bin").write_bytes(bytes(range(256)) * 4)
PY

T=./spidev_test
D=/dev/spidev1.0

for speed in 100000 500000 1000000; do
    "$T" -D "$D" -s "$speed" -b 8 \
        -i tx-1024.bin -o "rx-$speed.bin"
    cmp -s tx-1024.bin "rx-$speed.bin"
    echo "speed=$speed cmp_rc=$?"
done
```

Ключ `-o` обязателен для побайтовой проверки: без него утилита печатает человекочитаемый дамп, а не сохраняет принятый поток. Успешный критерий для каждой частоты: код `spidev_test` равен 0, `cmp` возвращает 0 и размер RX равен 1024 байтам.

Проверьте устойчивость ста повторами на 1 МГц:

```
fail=0
for run in $(seq 1 100); do
    "$T" -D "$D" -s 1000000 -b 8 \
        -i tx-1024.bin -o rx-repeat.bin >/dev/null 2>&1 && \
        cmp -s tx-1024.bin rx-repeat.bin || fail=$((fail + 1))
done
echo "repeat_count=100 repeat_failures=$fail"
```

Прямая перемычка также позволяет проверить настройку CPOL/CPHA и порядка битов. Выполните передачи во всех четырёх режимах и с LSB-first:

```
"$T" -D "$D" -s 100000 -b 8 \
    -i tx-1024.bin -o rx-mode0.bin
"$T" -D "$D" -s 100000 -b 8 -H \
    -i tx-1024.bin -o rx-mode1.bin
"$T" -D "$D" -s 100000 -b 8 -O \
    -i tx-1024.bin -o rx-mode2.bin
"$T" -D "$D" -s 100000 -b 8 -H -O \
    -i tx-1024.bin -o rx-mode3.bin
"$T" -D "$D" -s 100000 -b 8 -L \
    -i tx-1024.bin -o rx-lsb.bin

for result in rx-mode0.bin rx-mode1.bin rx-mode2.bin \
    rx-mode3.bin rx-lsb.bin; do
    cmp -s tx-1024.bin "$result"
```

```

    echo "$result cmp_rc=$?"
done

```

Совпадение во всех режимах с прямой перемычкой проверяет применение настроек контроллером, но не означает, что реальная периферия поддерживает любой режим. CPOL, CPHA и порядок битов всегда выбирают по datasheet устройства.

Для наблюдения границы PIO/DMA выполните передачи длиной 64 и 65 байт и сравните счётчики до и после теста:

```

S=/sys/bus/spi/devices/spi1.0/statistics
grep -H . "$S"/messages "$S"/errors "$S"/timedout
grep -E '3002000.dma-controller|sun6i-spi' /proc/interrupts

dd if=tx-1024.bin of=tx-64.bin bs=64 count=1 status=none
dd if=tx-1024.bin of=tx-65.bin bs=65 count=1 status=none

"$T" -D "$D" -s 1000000 -b 8 -i tx-64.bin -o rx-64.bin
cmp -s tx-64.bin rx-64.bin
echo "length=64 cmp_rc=$?"
grep -E '3002000.dma-controller|sun6i-spi' /proc/interrupts

"$T" -D "$D" -s 1000000 -b 8 -i tx-65.bin -o rx-65.bin
cmp -s tx-65.bin rx-65.bin
echo "length=65 cmp_rc=$?"
grep -E '3002000.dma-controller|sun6i-spi' /proc/interrupts

grep -H . "$S"/messages "$S"/errors "$S"/timedout

```

Передача 64 байт должна увеличить счётчик IRQ sun6i-spi, не меняя IRQ DMA-контроллера. Передача 65 байт должна увеличить оба счётчика. Вместе с функцией sun6i_spi_can_dma() в исходном коде это подтверждает фактический переход от PIO к DMA, а статистика SPI подтверждает длины и отсутствие ошибок.

Проведите два отрицательных теста. Сначала при снятом питании убери­те перемычку, загрузите плату и убедитесь, что команда завершается, но cmp возвращает 1. Затем верните перемычку также без питания и за­просите аппаратный loopback:

```

"$T" -D "$D" -s 100000 -l -p '\xaa'
echo "internal_loop_rc=$?"
"$T" -D "$D" -s 100000 -b 16 -p '\xaa'
echo "bpw16_rc=$?"
"$T" -D /dev/spidev99.99 -s 100000 -p '\xaa'
echo "missing_device_rc=$?"

```

Ключ `-l` включает флаг `SPI_LOOP` внутри контроллера и не заменяет внешнюю перемычку. Для `spi-sun6i` этот режим и 16 бит на слово не поддерживаются, поэтому ожидаются диагностические сообщения и ненулевые коды завершения. Несуществующее устройство также должно дать ненулевой код.

Опасность повреждения

Устанавливайте и снимайте перемычку MOSI-MISO только при выключенном питании. Не соединяйте сигнальные выводы с питанием или землёй и не подавайте на них 5 В.

Проверка

Ожидаемый результат. При исправной перемычке каждый принятый байт совпадает с переданным. Несовпадение требует проверки имени `/dev/spidev*`, распиновки, режима, частоты и соединения MOSI-MISO.

Результаты проверочного стенда

На Lichee RV Dock с Linux 6.16.0-spi-lab PREEMPT_RT получено `/dev/spidev1.0`, драйвер `sun6i-spi` выделил RX- и TX-DMA-каналы. Внешний loopback прошёл со следующими результатами:

Проверка	Фактический результат
1024 байта, mode 0, 100/500/1000 кГц	сmp = 0 на каждой частоте
Длины 64 и 65 байт	обе передачи совпали
Режимы SPI 0-3	все совпали в электрическом loopback
MSB-first и LSB-first	оба порядка совпали
100 повторов по 1024 байта на 1 МГц	100 успешных, 0 несовпадений
Сводная статистика	110 сообщений, 110721 байт, 0 ошибок, 0 тайм-аутов
Разомкнутые MOSI/MISO	передача завершилась, сmp = 1
16 бит на слово, SPI_LOOP, отсутствующее устройство	ожидаемый ненулевой код

Одинаковый результат режимов 0-3 в тесте с прямой перемычкой не доказывает пригодность любого режима для реальной периферии: режим выбирают только по datasheet устройства.

```
[root@localhost my_spi_device]# python3 spi.py
SPI последовательный тест запущен...
Отправляю данные: 0, 1, 2, 3, ...
-----
Отправлено:  0 (0x00) | Получено:  0 (0x00) | Бинарно: 00000000
Отправлено:  1 (0x01) | Получено:  1 (0x01) | Бинарно: 00000001
Отправлено:  2 (0x02) | Получено:  2 (0x02) | Бинарно: 00000010
Отправлено:  3 (0x03) | Получено:  3 (0x03) | Бинарно: 00000011
Отправлено:  4 (0x04) | Получено:  4 (0x04) | Бинарно: 00000100
Отправлено:  5 (0x05) | Получено:  5 (0x05) | Бинарно: 00000101
Отправлено:  6 (0x06) | Получено:  6 (0x06) | Бинарно: 00000110
Отправлено:  7 (0x07) | Получено:  7 (0x07) | Бинарно: 00000111
Отправлено:  8 (0x08) | Получено:  8 (0x08) | Бинарно: 00001000
Отправлено:  9 (0x09) | Получено:  9 (0x09) | Бинарно: 00001001
Отправлено: 10 (0x0A) | Получено: 10 (0x0A) | Бинарно: 00001010
```

Рис. 4 – Результат полнодуплексного теста SPI с перемычкой между MOSI и MISO

Задание

1. Проанализируйте Device Tree платы Lichee RV Dock и найдите описание SPI1 и группы выводов PD10–PD15.
2. Убедитесь, что выводы PD10–PD15 не заняты конфликтующим узлом.
3. Создайте отдельный DTS, активируйте SPI1 и добавьте дочерний узел для доступа через SPIDEV.
4. Включите CONFIG_SPI, CONFIG_SPI_MASTER, CONFIG_SPI_SUN6I, CONFIG_SPI_SPIDEV и CONFIG_DMA_SUN6I.
5. Соберите ядро, DTB и статическую утилиту spidev_test.
6. Добавьте отдельную загрузочную запись по PARTUUID, не удаляя known-good комплект.
7. Подтвердите наличие /dev/spidev*, binding драйверов и выделение DMA-каналов.
8. Проведите тесты на 100, 500 и 1000 кГц, в режимах 0–3, с LSB-first, на границе 64/65 байт и в 100 повторях.
9. Проведите отрицательные тесты без перемычки, с неподдерживаемыми SPI_LOOP и 16 битами на слово, а также с отсутствующим устройством.
10. Продемонстрируйте работу преподавателю.

Контрольные вопросы

1. Какие четыре сигнала используются в SPI и за что отвечает каждый?
2. Чем топология SPI отличается от шинной топологии I2C?
3. Почему узел SPI в Device Tree по умолчанию имеет status = "disabled"?

4. Зачем нужен SPIDEV и какое устройство появляется в /dev после его активации?
5. Какой идентификатор compatible используется в этой работе для привязки SPIDEV?
6. Когда драйвер spi-sunbi выбирает PIO, а когда DMA? Является ли DMA обязательной для инициализации контроллера?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- фрагменты Device Tree до и после изменения;
- включённые параметры конфигурации ядра;
- вывод проверки /dev/spidev* и значимые сообщения dmesg;
- схему или фотографию перемычки MOSI-MISO;
- таблицу частот, режимов, длин, числа повторов и ошибок полнодуплексного теста;
- изменение IRQ контроллера SPI и DMA на передачах 64 и 65 байт;
- результат отрицательных тестов и объяснение отличия внешней перемычки от SPI_LOOP;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.